

Plectis

Research with AI, mathematics and software

WILL COOK*

September 2026

I built Plectis to do as much useful research with AI as possible before asking an expert to spend their time on it. The aim is a continuing exchange: prepare the work so someone can interpret it, challenge it or give direction, do the next round, and come back with something worth their attention. The name draws on the Latin *plectere*, to weave or twist, because there is a lot to hold together: mathematics, software, a safety doctrine, an interface, and other people's contributions, across two repositories and a website.

My limited mathematical background made me a useful test case. How do you work responsibly with material you cannot assess yourself, but an expert could? I could observe where I needed help and watch those limits move as the models improved. That question came before the [Navier–Stokes announcement](#). As Terence Tao and his [co-signatories](#) argue, solving a problem is one part of mathematics; the ideas still have to be understood, developed and communicated, and for the foreseeable future that runs through human experts in each field.

Regardless of any slowdown, the writing is on the wall: the ceiling on the volume of work that can leave the dam of AI-generated mathematics per second keeps rising, throttled or not. If open models run six to twelve months behind the frontier, the floodgates get cheap quickly and everyone jumps on them. That leaves a little time for mathematics to prepare to surf rather than drown. Alongside the proposals for [shared frontier AI standards](#), I think mathematics needs a practical floor for AI-assisted work: intelligible arguments, appropriate checks, visible limitations and attribution. Experts should feel free to decline work below it and engage with work that meets it. Incentives move people where rules do not.

Plectis is one attempt to prototype that floor. The [mathematics repository](#) follows eight deliberately hard Erdős problems, and all eight remain open. It holds partial results, computations, Lean proofs, and Comparator checks of selected statements; I have prioritised exposition and interesting mathematics over formalising everything. Each problem has a [short paper](#) and a [long research record](#): the short one is for people, the long one is the full reasoning that a researcher or an agent can continue. Submitting to Palomar also produced a [merged fix](#) for a renderer bug other submitters had hit.

Failed routes are preserved just as much as a successful lemma. A later researcher, or a better model, may connect observations across the whole corpus, find another use for one, or ask a more interesting question. It is like sending people out on random walks over new ground and keeping every report: someone falls over, so there is a hill; there is a hill, so there may be water. I want a shared map of the ground explored, not one route to one summit.

The same difficulty runs through everything I have built: building has been easier than showing people what was built. The [website](#), the [glossary](#) and the linked papers let a reader choose their depth and follow through to the underlying work, and the [front end, shown on YouTube](#), connects maps, code, agent activity and records of what ran. I want better models to help keep human understanding intact as systems grow, including making organisations built around agents easier to audit.

The [software repository](#) releases selected, testable components, because I could not see how to release the whole private system responsibly. Its [doctrine](#) sets out the axioms, principles and anti-principles that govern what agents build and how they build it, enforced through checks and permissions where I can. The [questions I posed in June](#), on how behaviour changes with capability and environment, how a builder gets independent scrutiny, and what agent-run organisations change, read as more immediate now.

The [systems paper](#) and the [open-source paper](#) explain how people can contribute direction, review, exposition, software or compute, and how credit stays attached as a contribution is used and extended rather than going only to whoever finishes a problem. This is a prototype and it needs independent scrutiny. I want an expert to contribute one useful insight without inheriting the whole workload, and for us to come back with work that makes their next contribution worth making.